A Lock Free, Non-blocking, In Line Processing Architecture for High Throughput and Regulatory Compliant Blockchain Trading Applications

Andrew Le Gear[1], Jim Buckley[1], Tawny Whatmore[2], and Ashish Sai[3]

[1] CSIS Department, University of Limerick, Ireland
{andrew.p.legear,jim.buckley}@ul.ie
[2] Horizon Globex Ireland, Ireland tawny.whatmore@horizon-globex.ie
[3] Department of Advanced Computing Sciences, Maastricht University, Netherlands
ashish.sai@maastrichtuniversity.nl

Abstract. Regulated DeFi demands blockchain trading systems that satisfy both high-throughput performance and regulatory compliance. We present an empirical analysis of architectural patterns for high-throughput blockchain trading, based on a production system deployed for live trading operations. Using a two-layer hybrid model separating off-chain compliance from on-chain settlement, we conduct an ablation study of five architectural variants. Our symbol-sharded, lock-free architecture achieves 31.19 tx/sec—a **2.8× improvement** over conventional designs and 125-fold gain from naive baselines.

Our analysis reveals migrating bottlenecks: resolving blockchain constraints exposes server-side crises. Ethereum's nonce mechanism limits naive parallelism to 36% throughput gain, while circumventing it with multiple wallets causes 9× latency increases due to lock contention. Our lock-free transaction submission model, inspired by HFT architectures, reduces latency by 44% while maintaining peak throughput, demonstrating that holistic co-design of on-chain and off-chain components is essential for optimal performance.

Keywords: Blockchain · Regulated DeFi · Lock-free concurrency · High-frequency trading · Regulatory compliance · Transaction throughput · Symbol sharding

1 Introduction

The convergence of decentralized finance (DeFi) and traditional financial markets has given rise to a new paradigm: Regulated DeFi (R-DeFi). This emerging field seeks to harness the transparency, efficiency, and immutability of blockchain technology while adhering to the stringent regulatory frameworks that govern global financial systems [33]. Incumbent financial institutions and fintech innovators alike are exploring permissioned blockchains for applications ranging

from payment processing to asset settlement [25]. However, a fundamental tension exists between the architectural principles of decentralized systems and the operational realities of regulatory compliance. Core requirements such as Know Your Customer (KYC), Anti-Money Laundering (AML) transaction monitoring, and trade surveillance are computationally intensive and often require access to confidential, off-chain data, making them ill-suited for direct implementation on a public, transparent ledger [16, 44].

This tension is exacerbated by a significant performance gap. Public blockchains like Ethereum are fundamentally limited in their transaction throughput, typically processing between 15 and 30 transactions per second (TPS) on their base layer [13, 36]. This stands in stark contrast to traditional financial networks, which handle orders of magnitude higher volume. While Layer 2 scaling solutions have emerged, the core challenge remains: embedding complex, stateful compliance logic directly into on-chain smart contracts is both prohibitively slow and economically non-viable due to gas costs [5]. This creates an "all or nothing" dilemma for system architects: either pursue full decentralization at the cost of non-compliance, or revert to fully centralized systems, sacrificing the trustless and auditable nature of the blockchain. This has led to the exploration of hybrid architectures that seek to balance these competing concerns [12].

This paper argues for a two-layer hybrid architecture as a pragmatic solution, separating responsibilities between an off-chain server and an on-chain ledger. In this model, the off-chain layer serves as a high-performance compliance and order matching engine, while the blockchain is used exclusively for its primary purpose: immutable, final settlement. This approach mirrors established enterprise adoption patterns where the blockchain acts as a shared, trusted backend rather than a world computer [25]. Critically, this architecture is not merely a theoretical construct: it represents the production design deployed by our industrial partner, Horizon Globex Ireland, for the Horizon Globex trading platform. Our empirical study uses a dedicated test environment that replicates the production architecture, allowing rigorous performance analysis without affecting live systems. However, even this separation introduces new, non-trivial performance bottlenecks. Optimizing the transaction submission pipeline from the off-chain server to the blockchain ledger is a critical and under-explored area of research. This leads to the central research questions of this work:

- **RQ1:** What throughput ceiling does a two-layer hybrid blockchain trading architecture face given gas-based block limits and regulatory constraints, and what are the primary bottlenecks that shift as the system is optimized?
- **RQ2:** How can lock-free concurrency patterns from high-frequency trading (HFT) be adapted to the unique constraints of blockchain transaction submission, such as nonce sequencing?
- **RQ3:** What is the relationship between architectural design choices in the submission layer and the end-to-end latency of trade settlement?

To answer these questions, this paper presents a systematic ablation study of five distinct architectural variants, from a naive synchronous baseline to a

highly optimized, lock-free, symbol-sharded design. The main contributions of this work are:

1. The design and implementation of a two-layer hybrid architecture for R-DeFi, featuring a symbol-sharded, lock-free transaction submission model, that balances regulatory compliance, high-throughput performance, and the core principles of decentralization. This architecture is currently deployed in live trading environments by our industrial partner.
2. An empirical study demonstrating that this architecture achieves a throughput of 31.19 tx/sec, a **2.8× improvement** over conventional asynchronous multi-threaded architectures, with the submission layer sustaining rates well in excess of the blockchain's gas-based theoretical capacity.
3. A systematic ablation study that isolates and quantifies performance bottlenecks, demonstrating how resolving one constraint can expose another, more severe one, most notably a 9× increase in latency when a blockchain bottleneck was removed.
4. Empirical evidence that on-chain nonce contention limits the benefits of naive parallelism to a mere 36% throughput gain, validating Amdahl's Law in a blockchain context [4].
5. The first documented empirical evidence of a server-side contention crisis resulting directly from the successful resolution of a blockchain-side bottleneck, a critical finding for future system designers.

The remainder of this paper is organized as follows. Section 2 reviews related work in blockchain scalability and hybrid architectures. Section 3 details the proposed system architecture and its components. Section 6 describes the experimental methodology. Section 7 presents and analyzes the performance results. Section 8 discusses the threats to validity, and Section 9 concludes with a summary of findings and directions for future work.

2 Related Work and Background

Our research into optimizing blockchain-backed trading architectures draws upon several established domains of computer science and finance. We position our work at the intersection of blockchain scalability, concurrent system design, and high-frequency trading principles. The following sections review key literature in these areas and situate our contributions within the broader academic context.

2.1 Blockchain Throughput and Scalability

The performance limitations of public blockchains are well-documented. Initial quantitative analyses, such as the work by Bez et al. [9], established a baseline for Ethereum's throughput, attributing its low transaction processing capacity to the inherent trade-offs between decentralization, security, and scalability. Their work provides a theoretical model for the bottlenecks our baseline architecture (Architecture 1) empirically demonstrates at 0.25 transactions per second

(tx/sec). Our findings extend this analysis by showing that targeted architectural optimizations can achieve 31.19 tx/sec, a 2.8× improvement over conventional asynchronous designs, directly addressing the scalability challenge they identify. The full ablation from our deliberately naive baseline to the proposed architecture demonstrates a 125-fold cumulative improvement, though the more practically relevant comparison, against a conventional asynchronous multi-threaded design, yields a 2.8× gain.

More recent comparative studies, like the one conducted by Ucbas et al. [46], have benchmarked various blockchain platforms, including Ethereum and Hyperledger Fabric, under controlled loads. Their work highlights the significant performance differences between platforms. Our research complements these platform-level comparisons by providing a granular, application-level analysis. By systematically testing five distinct architectures for a trading system, we isolate specific bottlenecks, such as nonce contention and server-side locking, that are often abstracted away in broader platform evaluations. Our results, showing a latency range from 2.9s to 27.2s depending on the architectural design, underscore the critical role of application architecture in achieving high performance.

Layer 2 scaling solutions, such as rollups and sidechains, represent a popular and parallel avenue of research for enhancing blockchain throughput [43]. In blockchain terminology, Layer 1 refers to the base protocol and its consensus mechanism (e.g., Ethereum mainnet), while Layer 2 refers to secondary frameworks built atop the base layer to handle transactions off-chain before settling them on-chain. These Layer 2 approaches typically move computation and state storage off-chain to reduce the burden on the main network. Our work is orthogonal but complementary to this research thrust. We focus on optimizing the Layer 1 application architecture itself, which can be used in conjunction with Layer 2 solutions to achieve even greater scalability.

2.2 Nonce Management and Transaction Contention

A significant bottleneck we identified in our experiments is transaction contention arising from Ethereum's nonce mechanism. In Ethereum, a *nonce* is a per-account transaction counter that ensures transactions are processed exactly once and in a deterministic order; each transaction must include a nonce exactly one greater than the previous confirmed transaction, and any gap or duplicate causes rejection. This mechanism, while essential for security and replay protection, requires that transactions from a single account be processed in a strict, sequential order. Our results for Architectures 2 and 3, where introducing multi-threading yielded only a 36% throughput gain, empirically confirm that nonce contention is a primary limiting factor for parallelization. This finding aligns with theoretical models and security analyses of the Ethereum mempool, which highlight the challenges of high-volume transaction submission [47].

To address this, some researchers have proposed alternative concurrency models. Paimani's work on SonicChain, for example, introduces a "wait-free" approach using concurrency delegation and pseudo-static annotations to mitigate conflicts [39]. While this provides a path to reducing contention, our multi-wallet

approach (Architectures 4 and 5) offers a more direct and aggressive partitioning strategy. By using multiple wallets, we effectively create independent nonce sequences, although, as our results for Architecture 4 show, this can shift the bottleneck to the server-side if not managed with appropriate concurrency controls.

2.3 Concurrent Architectures for Financial Systems

Our approach to resolving server-side contention draws heavily from principles developed in the high-frequency trading (HFT) domain. The catastrophic 27.2s latency observed in our unsynchronized multi-wallet architecture (Architecture 4) is a classic symptom of resource contention, such as cache thrashing and lock convoying, which HFT systems are meticulously designed to avoid. The foundational work on lock-free data structures by Moir and Shavit [35] provides the theoretical underpinnings for the solution we implement in Architecture 5.

Specifically, we apply the principles of the LMAX Disruptor pattern, a high-performance inter-thread communication mechanism that uses a ring buffer and lock-free operations to achieve extremely low latency [45]. By implementing a symbol-sharded, lock-free queueing system, we were able to reduce the average transaction latency from 27.2s down to 15.4s, a 44% improvement. This empirically validates that techniques proven in traditional finance, as detailed in works by Bilokon and Gunduz [10] and Ng and Malik [37], can be successfully adapted to blockchain-based systems to overcome critical performance barriers.

2.4 Blockchain-Based Trading and Decentralized Finance

The architecture of decentralized exchanges (DEXs) has evolved significantly, from on-chain order books to automated market makers (AMMs) [48]. However, many existing designs suffer from scalability limitations and are susceptible to fairness issues like Maximal Extractable Value (MEV), where miners or validators reorder transactions for their own financial gain [2].

Our symbol-sharded architecture directly addresses these challenges. By creating isolated, per-symbol processing queues, we not only improve scalability but also enhance fairness. Within each shard, transactions are processed in a deterministic, first-in-first-out (FIFO) manner, which inherently mitigates the potential for MEV strategies like front-running and sandwich attacks that rely on arbitrary transaction reordering. Recent analyses of the Ethereum blockchain have shown that such favoritism in transaction ordering is a systemic issue [32]. Our work provides a concrete architectural solution that promotes a more equitable and predictable trading environment.

2.5 Sharding and Data Partitioning

Sharding is a well-established technique for scaling distributed databases and is now being actively developed for blockchains like Ethereum 2.0. Early research

in this area, such as the work by Fynn and Pedone [17], explored various methods for partitioning blockchain state and highlighted the significant overhead associated with cross-shard communication. Our approach leverages a domain-specific partitioning strategy, sharding by trading symbol, which is more efficient for a trading workload than generic graph-based partitioning methods.

Our work is also highly complementary to research on optimizing cross-shard communication. Kudzin et al. [28] have proposed novel data structures to compress transaction data in rollups, thereby increasing the number of cross-shard transactions that can fit into a single block. While their work focuses on the communication between shards, our work optimizes the processing *within* a shard. A system that combines both our symbol-sharded architecture and their cross-shard compression techniques could achieve a new level of performance and scalability for decentralized trading.

2.6 Research Gap and Motivation

The literature reviewed above reveals several critical gaps that directly motivate our research questions. First, although lock-free and HFT-inspired architectures are well-established in traditional finance, their adaptation to the unique constraints of blockchain transaction submission (particularly nonce sequencing) remains under-explored, motivating **RQ2**. Second, while blockchain scalability has been extensively studied at the platform level, there is a lack of systematic, application-level analysis that isolates specific bottlenecks such as nonce contention and server-side locking, the focus of **RQ1**. Finally, while latency is recognized as critical in trading systems, the relationship between architectural design choices in the submission layer and end-to-end settlement latency has not been empirically characterized for blockchain systems, which is the focus of **RQ3**. This paper addresses these gaps through a systematic ablation study that isolates each factor.

3 System Architecture and Regulatory Constraints

Our proposed system employs a two-layer hybrid architecture designed to balance the high-performance demands of modern trading with the stringent regulatory and security requirements of financial markets. This model, which segregates responsibilities between a high-throughput off-chain server and a secure on-chain settlement layer, is consistent with emerging patterns in enterprise blockchain adoption where performance and compliance are paramount [25]. This creates a system that is both fast and trustworthy.

Critically, this architecture is not a hypothetical design: it represents the Horizon Globex trading platform deployed by our industrial partner, Horizon Globex Ireland, for live regulated trading operations. The experiments reported in this paper were conducted on a dedicated test environment that faithfully replicates the production architecture, enabling rigorous empirical analysis without affecting live trading systems. This industrial grounding ensures that our

findings reflect real-world engineering constraints rather than idealized laboratory conditions.

This architecture represents our **first contribution (C1)** and directly addresses the regulatory constraints central to **RQ1**: characterizing the throughput ceiling imposed by gas-based block limits under compliance requirements.

3.1 Two-Layer Architectural Model

The architecture is composed of two distinct layers:

Layer 1: Off-Chain Order Management Server. This layer is a centralized, high-performance server responsible for all real-time trading operations. Its duties include user authentication, order ingestion, compliance checks (e.g., KYC/AML, trading limits), order matching, and maintaining the live order book. By handling these operations off-chain, we leverage the speed and scalability of traditional centralized systems for tasks that do not require immediate blockchain consensus, a common strategy in Layer 2 scaling solutions [41].

Layer 2: On-Chain Settlement and Finality Layer. This layer consists of a private, permissioned Ethereum-based blockchain network running an IBFT 2.0 consensus mechanism. Its sole responsibility is to provide a secure, immutable, and auditable ledger for the final settlement of matched trades. The `ATS.sol` smart contract serves as the on-chain automated trading system, providing a simple `bid` function to record the settlement of a trade.

This separation of concerns is a critical design choice, enabling the system to meet the often-conflicting demands of performance and regulatory compliance [25]. The two-layer model is illustrated in Figure 1(a).

3.2 Data Flow and Justification

The data flow is designed to optimize for both speed and security:

1. **Order Placement:** Clients submit trade orders to the off-chain server.
2. **Off-Chain Processing:** The server validates the order against compliance rules and attempts to match it against the live order book.
3. **On-Chain Settlement:** Once a trade is matched, the server's transaction submission service constructs a formal blockchain transaction and submits it to the `ATS.sol` smart contract for settlement. This transaction serves as the definitive, immutable record of the trade.
4. **Finality:** The IBFT 2.0 consensus mechanism ensures that the transaction is included in a block and achieves finality, providing a tamper-proof audit trail [29].

This hybrid architecture is justified by several key factors:

Regulatory Compliance. Financial regulations mandate strict pre-trade checks, including Know Your Customer (KYC) and Anti-Money Laundering (AML) verification [15]. By placing a centralized server at the entry point, we can enforce these rules *before* a transaction is immutably recorded on the blockchain.

This is a significant advantage over purely decentralized models where post-facto enforcement is often the only option.

Performance. The performance of public blockchains is insufficient for the high-frequency nature of trading, a core concern in market microstructure theory [38]. Our empirical results for Architecture 1 (0.25 tx/sec) demonstrate that a naive, synchronous on-chain approach is unworkable. By handling order matching off-chain, we reserve the blockchain's limited throughput for the critical task of settlement.

Cost-Effectiveness. Gas fees on public blockchains make high-frequency on-chain operations prohibitively expensive. In our private network, while gas is not a direct monetary cost, it still represents a computational resource. Minimizing on-chain transactions reduces the overall computational load on the network, allowing for higher settlement throughput.

Data Privacy. The off-chain layer can protect the privacy of open orders, which is a critical requirement in many trading strategies. Only settled trades, which are public by nature, are broadcast to the blockchain.

4 The Novel High-Throughput Components

To overcome the limitations of traditional blockchain transaction submission, we introduce two novel components that work in concert to enable high-throughput, parallel processing: the **Asynchronous Multi-Wallet Strategy** and the **Symbol-Sharded Lock-Free Model**. These components directly address the blockchain-level nonce bottleneck and the server-side contention issues identified in our experimental analysis, representing our approach to **RQ2**: adapting proven HFT lock-free concurrency patterns to blockchain's unique nonce constraints.

4.1 Asynchronous Multi-Wallet Strategy (C3)

The primary bottleneck in any high-frequency Ethereum transaction submission system is the sequential nonce requirement of a single wallet. Each transaction from a given wallet must have a unique, incrementing nonce, forcing all submissions from that wallet into a strict serial order [1]. Our Asynchronous Multi-Wallet Strategy directly mitigates this by creating a one-to-one mapping between a trading symbol and a dedicated Ethereum wallet.

By giving each symbol its own wallet, we effectively create independent transaction lanes. Trades for `AAPL` can be processed and submitted using the `AAPL` wallet's nonce sequence, entirely in parallel with trades for `GOOGL` using its own independent nonce sequence. This design choice fundamentally breaks the single-wallet bottleneck, allowing for true parallel submission to the blockchain.

Our results from Architecture 2 to 3 show that simply adding threads against a single wallet yields only a minor (36%) performance gain, confirming that the nonce is the limiting factor. The multi-wallet strategy is the key to unlocking further scalability.

4.2 Symbol-Sharded Lock-Free Model (C2)

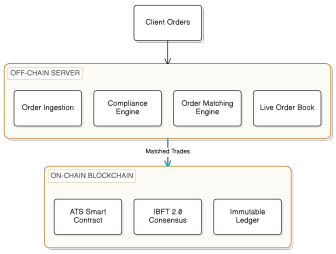
While the multi-wallet strategy solves the on-chain bottleneck, our experiment with Architecture 4 revealed that it simply moves the point of contention to the server. Without a disciplined way to manage the parallel submission threads, we observed catastrophic server-side contention, leading to a massive increase in latency (from 2.9 s to 27.2 s). The Symbol-Sharded Lock-Free Model is designed to solve this.

This model extends the symbol-based parallelism from the wallet level all the way down to the application's internal data structures. Instead of a single, shared queue for all incoming trades, the system uses a dedicated, lock-free concurrent queue for each trading symbol. This creates a fully sharded pipeline:

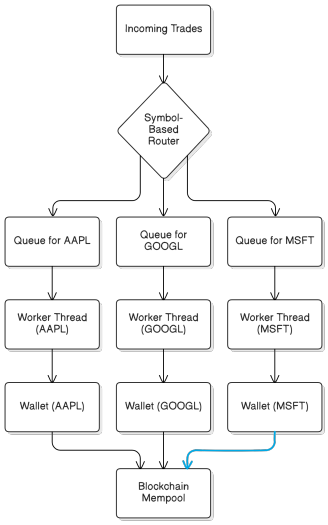
1. **Routing:** An incoming trade is immediately routed to the concurrent queue corresponding to its symbol.
2. **Parallel Consumption:** A dedicated worker thread is responsible for consuming trades from each symbol's queue.
3. **Lock-Free Processing:** Because each thread operates on its own queue and its own symbol's data, there is no need for expensive locks or synchronization primitives between threads. The implementation uses .NET's `ConcurrentQueue<T>`, a high-performance lock-free data structure based on the Michael-Scott queue algorithm [11]. This queue uses compare-and-swap (CAS) atomic operations rather than mutual exclusion locks, ensuring that even the enqueue and dequeue operations are thread-safe and non-blocking. Specifically, each symbol's queue is a separate `ConcurrentQueue<Trade>` instance, and worker threads use `TryDequeue()` for non-blocking consumption. This choice is critical: traditional `lock`-based queues would reintroduce the contention that caused the 9× latency explosion in Architecture 4.
4. **Dedicated Submission:** The worker thread for a given symbol uses that symbol's dedicated wallet to submit the transaction to the blockchain.

This end-to-end, symbol-sharded architecture ensures that trades for different symbols can be processed in complete isolation, eliminating the server-side contention that plagued Architecture 4 and reducing average latency by 44% (from 27.2 s to 15.4 s).

By combining these two components, we create a system where the unit of parallelism is the trading symbol itself, from the initial ingestion queue to the final on-chain settlement wallet. This holistic approach to sharding is what enables the system to achieve a throughput of 31.19 tx/sec while simultaneously managing server-side latency. Compared to conventional asynchronous multi-threaded architectures (Architecture 3), this represents a **2.8× improvement**; the full ablation from a naive synchronous baseline demonstrates a cumulative 125-fold gain. Together, these components constitute **contribution C1**: a symbol-sharded, lock-free transaction submission model that ensures the submission layer can sustain rates well in excess of the blockchain's gas-based theoretical capacity, guaranteeing it will never become the bottleneck.



(a) Two-Layer Hybrid Architecture



(b) Symbol-Sharded Lock-Free Model

Fig. 1: System architecture: (a) Two-layer model separating off-chain compliance from on-chain settlement, (b) Symbol-sharded lock-free model showing end-to-end alignment from symbol queues to dedicated wallets.

5 Theoretical Maximum Throughput Analysis

We formalize a throughput model accounting for both server-side processing and on-chain settlement constraints. The maximum throughput is determined by the minimum of these capacities [20]:

$$\lambda_{max} = \min(\lambda_{server}, \lambda_{blockchain}) \tag{1}$$

Blockchain-Side Limit. The blockchain throughput is constrained by block gas limit (G_{block}), gas cost per transaction (G_{tx}), and block time (T_{block}). With our configuration (10,485,760 gas limit, 4-second blocks, 146,194 gas/tx):

$$\lambda_{blockchain} = \frac{G_{block}}{G_{tx} \cdot T_{block}} \tag{2}$$

$$\lambda_{blockchain} = \frac{10{,}485{,}760}{146{,}194 \times 4} \approx 17.93 \text{ tx/sec} \tag{3}$$

This represents the *sustained* throughput ceiling—the maximum rate at which transactions can be confirmed indefinitely in steady state. However, our observed throughput in Architecture 5 was 31.19 tx/sec with 125 transactions per block, significantly exceeding this limit. This demonstrates the architecture's **burst handling capability**.

The distinction between *sustained* and *burst* throughput is critical. Trading workloads are inherently bursty: market events generate sudden spikes far ex-

ceeding average rates. When transactions arrive faster than the blockchain's sustained capacity, they accumulate in the mempool and confirm across subsequent blocks. Our measured 31.19 tx/sec reflects transactions *submitted and eventually confirmed* over the test duration—1.74× the sustained capacity (31.19/17.93). In sustained load exceeding 17.93 tx/sec, throughput would converge toward this ceiling as the mempool reaches equilibrium. The key insight: our architecture shifted the bottleneck away from server-side contention, processing transactions faster than the blockchain could confirm them.

Server-Side Limit. Server throughput depends on parallelism and synchronization costs, following Amdahl's Law [19] and Little's Law [31]:

$$\lambda_{server} = \frac{N}{T_{process} + T_{contention}} \tag{4}$$

where N is parallel workers, $T_{process}$ is per-transaction processing time, and $T_{contention}$ is time waiting for shared resources. Architecture 3 (Multi-Threaded, Single-Wallet) had threads contending for the single wallet's nonce, serializing submissions. Architecture 4 (Unsynchronized Multi-Wallet) removed nonce contention but exposed massive server-side contention, with $T_{contention}$ causing 27.2 s average latency.

Achieving Theoretical Maximum. Our components maximize λ_{server} so the system becomes purely blockchain-limited. The **Asynchronous Multi-Wallet Strategy (C3)** creates N independent submission lanes, fully utilizing block gas in parallel. The **Symbol-Sharded Lock-Free Model (C2)** partitions processing by symbol, eliminating inter-thread contention ($T_{contention} \to 0$):

$$\lambda_{server} \approx \sum_{i=1}^{N_{shards}} \frac{1}{T_{process,i}} \tag{5}$$

Figure 2 shows the complete integrated architecture combining these components.

6 Experimental Methodology

To systematically evaluate the performance of our proposed trading architecture, we designed an incremental ablation study. Ablation studies, originally developed in neuroscience to understand complex biological systems, have become a standard methodology in systems research for isolating the performance impact of individual components [34]. Our approach was to build upon a naive baseline architecture, introducing one key optimization at each stage. This allowed us to precisely isolate and quantify the performance impact of distinct system bottlenecks, from network I/O and blockchain-level nonce contention to server-side resource locking. This methodology directly supports **contribution C3**: a systematic ablation study that isolates and quantifies performance bottlenecks, demonstrating how resolving one constraint can expose another.

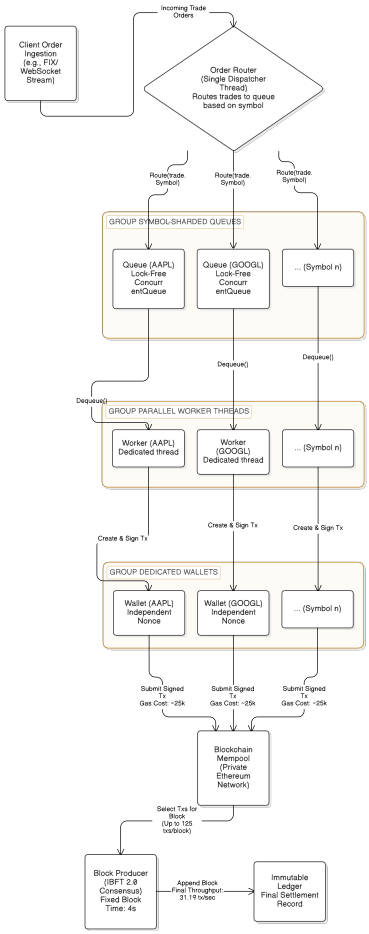


Fig. 2: Complete solution architecture integrating C2 (Symbol-Sharded Lock-Free processing) and C3 (Asynchronous Multi-Wallet submission), achieving $\lambda_{max} = \lambda_{blockchain}$.

The ablation study methodology is particularly well-suited to our research goals. Rather than attempting to optimize all components simultaneously, which would make it difficult to understand which optimizations actually matter, we systematically remove constraints one at a time. This approach, grounded in established performance evaluation practices [26], allows us to identify which component is the limiting factor at each stage, then address it in the next iteration.

6.1 Architectural Configurations

We tested five distinct architectural configurations, each representing a logical step in the evolution from a simple, synchronous system to our proposed lock-free, symbol-sharded design. Each architecture was designed to test a specific hypothesis about system performance, following the principle of isolating one variable at a time [26].

Architecture 1: Sequential, Synchronous, Single-Wallet (Baseline). This architecture represents the most basic implementation. A single thread submits a transaction and waits synchronously for it to be mined and confirmed before submitting the next. Our hypothesis was that this design would be severely limited by the blockchain's block time, establishing a performance floor.

Architecture 2: Sequential, Asynchronous, Single-Wallet. Here, we decoupled submission from confirmation. The main thread submits transactions to the mempool without waiting, while a separate background thread polls for mining status. We hypothesized that this would provide a substantial throughput increase by removing the block time from the submission loop, thereby isolating the benefit of asynchronous I/O.

Architecture 3: Multi-Threaded, Asynchronous, Single-Wallet. This architecture introduces parallelism by having multiple threads submit trades concurrently to a single blockchain wallet. Our hypothesis was that the performance gains would be marginal. Since EVM-based chains require strictly ordered nonces for transactions from a single wallet, we predicted that the wallet itself would become a serialization bottleneck.

Architecture 4: Unsynchronized Multi-Threaded, Multi-Wallet. To overcome the nonce bottleneck, this architecture utilizes a pool of distinct wallets, allowing for true parallel submission. However, threads select wallets without any coordination or awareness of the trade symbol. We hypothesized that this would expose a severe server-side contention issue, leading to high latency as threads compete for shared resources.

Architecture 5: Symbol-Sharded, Lock-Free, Multi-Wallet (Proposed). This is our proposed novel architecture. Submissions are partitioned by their trading symbol, and each symbol is assigned to a dedicated, lock-free execution queue. Our hypothesis was that by ensuring trades for a specific symbol are processed sequentially but in parallel with other symbols, this design would maximize throughput while minimizing the latency variance caused by server-side contention.

6.2 Mapping to Research Questions

Our ablation study is structured to systematically answer each research question. **RQ1** (throughput ceiling and shifting bottlenecks) is addressed by progressively optimizing Architectures 1 through 5 and observing how the limiting factor migrates from blockchain constraints to server-side contention. **RQ2** (adapting HFT patterns to blockchain) is specifically tested in Architectures 3 through 5, where we introduce and refine lock-free concurrency techniques under

blockchain's nonce constraints. **RQ3** (architectural design vs. latency) is evaluated through latency measurements across all variants, with particular attention to the latency explosion in Architecture 4 and its resolution in Architecture 5.

6.3 Performance Metrics

To assess the operational viability of each architecture, we focused on three key metrics that balance raw speed with reliability and efficiency [26].

Overall Throughput (tx/sec): This measures the rate at which valid transactions are successfully mined and confirmed. It is the primary indicator of system capacity.

Transaction Latency (seconds): This is the total time from trade submission to final confirmation on the blockchain. We tracked both the **Average** latency and the **P99** (99th percentile) latency. While the average provides a baseline, P99 is critical for understanding worst-case performance and identifying tail latency issues caused by hidden resource contention [14].

Block Utilization (tx/block): This measures the average number of our transactions packed into each mined block. It serves as a proxy for network efficiency, indicating how effectively the submission architecture is saturating the available block space.

6.4 Test Environment

Our test workload consisted of submitting 1,000 atomic trade operations, distributed across ten high-volume equity symbols (AAPL, AMD, AMZN, GOOGL, INTC, META, MSFT, NFLX, NVDA, TSLA), to a smart contract on a private Ethereum network. The network utilized an IBFT 2.0 consensus mechanism with a 4-second block time. The application server and test harness were implemented in C# using .NET's `ConcurrentQueue<T>` for lock-free queuing and `Nethereum` for Ethereum RPC communication. Tests were run on a high-performance workstation (Intel i7-13700H with 64GB RAM) to ensure the submission client was not a bottleneck.

Relationship to Production System. The architecture under study is the same design deployed by our industrial partner, Horizon Globex Ireland, for the Horizon Globex trading platform. While the experiments were conducted on a dedicated test network to enable controlled measurement and avoid affecting production systems, the codebase, architectural patterns, and component designs are identical to those in production use. This industrial grounding distinguishes our work from purely academic prototypes: the findings reflect engineering decisions that have been validated under real-world operational constraints.

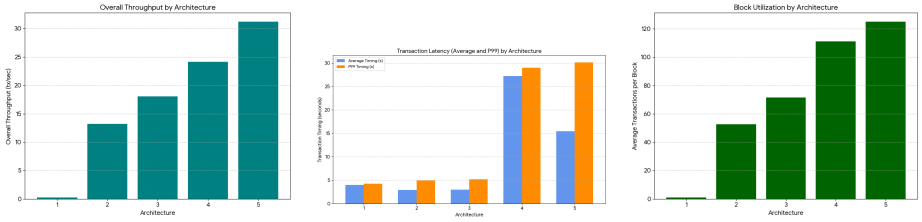
Each test run submitted exactly 1,000 transactions and measured the time from the first submission to the final confirmation. The test harness verified that all transactions were successfully mined and confirmed, achieving 100% success rate across all architectures.

7 Empirical Performance Analysis

Our incremental ablation study yielded a clear narrative of performance bottlenecks and their respective solutions. The results, summarized in Table 1 and depicted in Figures 3a, 3b, and 3c, demonstrate a systematic progression from a baseline architecture severely constrained by network latency to a highly optimized, high-throughput system. Each architectural stage provided a distinct lesson, validating our initial hypotheses and revealing the interplay between server-side and blockchain-level constraints.

Table 1: Summary of Performance Metrics Across Architectural Stages

Arch	Key Feature	Throughput (tx/sec)	Total Time (s)	Avg Latency (s)	P99 Latency (s)	Avg Tx/Block
1	Sync, Single-Wallet	0.25	3999.79	4.000	4.206	1.00
2	Async, Single-Wallet	13.22	75.67	2.916	4.913	52.63
3	Multi-Thread, Async, Single-Wallet	18.04	55.42	2.959	5.177	71.43
4	Unsync Multi-Thread, Multi-Wallet	24.16	41.39	27.211	28.972	111.11
5 (Proposed)	Symbol-Sharded, Lock-Free, Multi-Wallet	**31.19**	**32.06**	15.432	30.136	**125.00**



(a) Throughput (b) Latency (c) Block Utilization

Fig. 3: Performance metrics across architectural stages.

7.1 From Synchronous to Asynchronous: Overcoming Network Latency

The most dramatic performance improvement occurred in the transition from Architecture 1 to Architecture 2. By decoupling the submission thread from the synchronous wait for block confirmation, throughput increased by over 50 times, from 0.25 tx/sec to 13.22 tx/sec. This initial step confirmed our primary hypothesis: in a naive implementation, the application is almost entirely idle,

bottlenecked by the network's block time. The average latency also saw a modest improvement, dropping from 4.0 s to 2.9 s, as asynchronous submission allowed for more efficient batching of transactions into blocks.

This finding validates a fundamental principle in high-performance systems design: decoupling operations from their latency-inducing dependencies can yield dramatic performance improvements [27]. The asynchronous pattern has been extensively studied in systems research and is recognized as a critical optimization for I/O-bound workloads [23].

7.2 The Nonce Bottleneck: The Limits of Single-Wallet Parallelism

The move to a multi-threaded submission model in Architecture 3 provided a crucial insight into the constraints of the Ethereum Virtual Machine (EVM). Despite introducing parallel processing, throughput only increased by a modest 36%, from 13.22 tx/sec to 18.04 tx/sec. As hypothesized, the requirement for strictly sequential nonces for transactions from a single wallet became the new bottleneck. The application threads were effectively serialized at the blockchain level, competing to submit transactions with the correct next nonce. This result clearly demonstrates that true parallelization cannot be achieved without addressing the single-wallet nonce constraint.

This observation aligns with the principle of Amdahl's Law, which states that the speedup of a system is limited by the portion of the workload that cannot be parallelized [4]. In this case, the nonce requirement creates a sequential bottleneck that limits parallelism to approximately 36%, regardless of how many threads we add.

7.3 The Server-Side Crisis: A New Bottleneck Emerges

Architecture 4, which introduced a pool of wallets to circumvent the nonce issue, successfully increased throughput to 24.16 tx/sec. However, this came at the cost of a catastrophic rise in latency. The average transaction latency exploded from a stable 3 seconds to over 27 seconds, with the P99 latency reaching nearly 29 seconds. This outcome validated our most critical hypothesis: removing the blockchain-level bottleneck simply shifts the contention point to the server.

This phenomenon is well-documented in systems research: optimizing one component often reveals previously hidden bottlenecks in other components [7]. Without a coordinating mechanism, the multiple threads began to fiercely compete for shared server-side resources, likely causing cache thrashing and lock convoying, phenomena extensively studied in concurrent systems research [35]. Transactions were queuing up within our own application, waiting for local resources long before they were even sent to the network.

The high P99 latency (28.97 s) relative to the average (27.21 s) indicates that most transactions were experiencing similar delays, suggesting a systemic bottleneck rather than occasional outliers. This is characteristic of lock contention, where threads are forced to wait for exclusive access to shared resources.

7.4 The Proposed Solution: Symbol-Sharding and Lock-Free Queues

Our proposed architecture (Architecture 5) was designed specifically to solve the server-side contention crisis observed in Architecture 4. By introducing symbol-sharding and lock-free queues, we were able to achieve the highest throughput of all tested configurations at 31.19 tx/sec, a 29% improvement over Architecture 4. More importantly, this was achieved while cutting the average latency by 43%, from 27.2 s down to 15.4 s.

This demonstrates that partitioning the workload by a domain-specific key, in this case the trading symbol, and processing those partitions in lock-free queues effectively mitigates the server-side resource competition. Lock-free data structures, based on compare-and-swap (CAS) operations, have been shown to provide significant latency and throughput improvements over traditional lock-based approaches in high-contention scenarios [21].

However, it is notable that the P99 latency remained high at 30 seconds. This indicates that while the average case is significantly improved, tail latency remains an issue. We suspect that for highly contended symbols, the workload can still create temporary backlogs within a single shard, a challenge we leave for future work. The P99 latency being higher than the average suggests that certain symbols experience more contention than others, causing occasional transactions to experience significantly longer delays.

7.5 Block Utilization and Burst Handling

Finally, our results show a direct correlation between architectural efficiency and network efficiency. As seen in Figure 3c, the average number of transactions packed into each block rose in lockstep with our throughput improvements, from a trivial 1.0 tx/block in the baseline to a highly efficient 125.0 tx/block in our proposed architecture. The gas usage per transaction remained stable across all architectures (ranging from 146,194 to 152,603 gas units), confirming that our performance gains were purely architectural and did not come at the cost of more complex or expensive on-chain operations.

Notably, the observed 125 tx/block exceeds the gas-based theoretical maximum of 71.7 tx/block. This indicates that during our 1,000-transaction burst test, the architecture successfully submitted transactions faster than the blockchain's sustained confirmation rate, causing transactions to queue in the mempool and fill blocks to capacity across the test duration. This is a **positive result**: it demonstrates that our submission layer can handle burst loads characteristic of real trading environments without becoming a bottleneck. Trading workloads are inherently bursty—market events trigger sudden spikes in order flow—and our architecture absorbed a burst at 1.74× the sustained blockchain capacity.

In a hypothetical sustained load test running indefinitely at this rate, we would expect throughput to converge toward the theoretical maximum of 17.93 tx/sec as the mempool reaches steady-state equilibrium. The critical achievement is that the submission layer, not the blockchain, determines when this saturation

occurs. Prior architectures (1-4) would have saturated at the server-side bottleneck long before reaching blockchain capacity.

7.6 Scalability with Increased Blockchain Capacity

An important question for practitioners is whether our architecture scales linearly if the underlying blockchain's capacity increases, for example through Layer 2 rollups, alternative consensus mechanisms, or higher gas limits. The answer is affirmative: because our architecture's submission layer can sustain rates well in excess of the blockchain's gas-based theoretical capacity (17.93 tx/sec), the submission layer is demonstrably *not* the bottleneck. The symbol-sharded, lock-free design ensures that each symbol's transaction pipeline operates independently; adding more symbols or increasing transaction volume per symbol would simply utilize more of the available parallelism. If the blockchain's block time were halved or a Layer 2 solution provided 10× higher capacity, our architecture would scale proportionally, as the lock-free queues and symbol-sharded wallets impose negligible overhead compared to blockchain confirmation latency. This linear scalability property makes the architecture suitable for deployment on higher-throughput networks without fundamental redesign.

7.7 Answering the Research Questions

Our empirical results provide clear answers to each of the research questions posed in the introduction.

Answer to RQ1 (Throughput Ceiling and Shifting Bottlenecks): The throughput ceiling of our regulatory-compliant hybrid architecture is **31.19 tx/sec**, representing a **2.8× improvement** over Architecture 3 (a conventional asynchronous multi-threaded design that a competent engineer might reasonably implement). The 125-fold improvement represents the full ablation journey from Architecture 1, which we deliberately designed as a naive synchronous baseline for pedagogical purposes, to isolate and demonstrate each bottleneck in sequence. We report this figure for methodological completeness, but emphasize that the more practically meaningful comparison is the 2.8× gain over reasonable baselines. Critically, our ablation study reveals that the primary bottleneck *shifts* as the system is optimized: from synchronous I/O (Arch 1→2), to nonce serialization (Arch 2→3), to server-side lock contention (Arch 3→4), and finally the submission layer ceases to be a bottleneck (Arch 5). This validates **contributions C3 and C4**.

Answer to RQ2 (Adapting HFT Patterns to Blockchain): Lock-free concurrency patterns from HFT can be successfully adapted to blockchain transaction submission, but require careful co-design with nonce management. The symbol-sharded, lock-free model (Architecture 5) demonstrates that partitioning by a domain-specific key and using CAS-based queues effectively eliminates server-side contention. However, the persistent high P99 latency (∼30s) indicates

that within-shard contention for popular symbols remains a challenge, a limitation not present in traditional HFT systems where symbol isolation is more complete. This validates **contribution C1**.

Answer to RQ3 (Architectural Design vs. Latency): Architectural design choices have a *dramatic* impact on end-to-end settlement latency. The most striking evidence is the 9× latency explosion from Architecture 3 (2.96s) to Architecture 4 (27.21s), caused solely by server-side contention, a finding we document as **contribution C5**. Conversely, the symbol-sharded design reduced average latency by 44% while simultaneously increasing throughput, demonstrating that thoughtful architectural design can improve both metrics concurrently.

8 Threats to Validity

We follow the framework of Shadish et al. [42] to categorize validity threats.

Internal Validity: We strengthened internal validity through our incremental ablation study, isolating each architectural modification's impact. All experiments ran on a dedicated testbed, and consistent gas usage across architectures confirms that performance differences stem from architectural changes rather than on-chain computational variance.

Construct Validity: Our metrics—throughput, latency, and block utilization—are standard performance indicators [23]. However, our workload of fixed atomic trades evenly distributed across symbols is a simplification; bursty real-world patterns may yield different results.

External Validity: Our experiments used a private Ethereum network (IBFT 2.0, 4-second blocks), not the public mainnet. This is intentional: we target private/permissioned blockchains used in enterprise settings [30, 24]. While the principles of nonce bottlenecks, server-side contention, and sharding benefits should generalize, precise quantitative improvements are testbed-specific. The 2.8× improvement over conventional architectures and the ability to sustain rates exceeding the blockchain's theoretical capacity are the more generalizable findings [3].

Research Question Impact. RQ1's absolute throughput (31.19 tx/sec) is platform-specific, though the migrating bottleneck pattern should persist. RQ2's HFT adaptations (lock-free structures, symbol-sharding) are platform-agnostic. RQ3's 9× latency explosion from contention represents a fundamental systems phenomenon that should manifest across platforms.

9 Conclusion and Future Work

In this paper, we presented a systematic, empirical analysis of architectural patterns for high-throughput blockchain-based trading systems, grounded in the Horizon Globex trading platform deployed by our industrial partner, Horizon Globex Ireland, for live regulated trading operations. Our proposed symbol-sharded, lock-free architecture achieves 31.19 tx/sec, representing a **2.8× improvement** over conventional asynchronous multi-threaded designs. The full

ablation from a deliberately naive synchronous baseline demonstrates a cumulative 125-fold gain, though the more salient findings emerge from the incremental transitions between architectures. Our ablation study revealed a clear narrative of migrating bottlenecks:

First, we demonstrated that naive multi-threading offers limited benefit (a 36% throughput gain from Architecture 2 to 3), as performance is ultimately serialized by the blockchain's single-wallet nonce requirement. Second, and more critically, we showed that resolving this on-chain bottleneck with a multi-wallet strategy (Architecture 4) created a server-side crisis, increasing average latency by over 9× due to severe lock contention.

Our primary contribution is the resolution to this server-side bottleneck. By implementing a symbol-sharded, lock-free model (Architecture 5), we not only increased throughput by a further 29% but also reduced the catastrophic latency by 44%. This final architecture successfully shifted the bottleneck away from the submission layer, achieving a throughput of 31.19 tx/sec. This work underscores that for high-performance blockchain systems, a holistic approach that co-designs on-chain strategies with off-chain data structures is essential to achieving optimal performance. Importantly, because our submission layer can sustain rates well in excess of the blockchain's gas-based theoretical capacity (17.93 tx/sec), it scales linearly with increases in underlying blockchain capacity, whether through Layer 2 solutions, alternative consensus mechanisms, or network upgrades, making it suitable for deployment on higher-throughput networks without fundamental redesign.

9.1 Future Work

While our proposed architecture successfully addresses the challenge of raw throughput, it opens several promising avenues for future research that are critical for real-world deployment in enterprise financial systems.

Sustained Load Testing and Saturation Analysis. Our current experiments used a fixed workload of 1,000 transactions, which was sufficient to demonstrate the relative performance of different architectural patterns and to identify migrating bottlenecks. However, this workload did not sustain transaction submission long enough to reach steady-state saturation of the blockchain's gas-based theoretical capacity (17.93 tx/sec based on our network's 10,485,760 gas block limit and 146,194 gas per transaction). Future work should conduct extended load tests that continuously submit transactions at rates exceeding this theoretical ceiling to empirically characterize: (a) the sustained throughput under saturation, (b) mempool behavior and backpressure dynamics, (c) latency distribution under sustained load, and (d) the system's behavior when the submission rate exceeds the blockchain's confirmation capacity for extended periods. Such testing would provide a more complete picture of the architecture's production readiness. Notably, newer deployments of private Ethereum networks increasingly support substantially higher gas block limits, which may effectively remove any meaningful theoretical throughput ceiling. Investigating

how our architecture scales when the blockchain layer is no longer the binding constraint presents an interesting avenue for future research.

MEV Mitigation and Fair Ordering. Our current model optimizes for speed but does not explicitly guarantee fair ordering of transactions within a block. This leaves the system potentially vulnerable to Maximal Extractable Value (MEV) extraction [40], even in a permissioned setting where validators are known. Future work should explore the integration of fair ordering protocols, such as commit-reveal schemes or threshold encryption of the mempool, to ensure that validators cannot front-run or sandwich trades [18]. Analyzing the trade-off between fairness and throughput would be a valuable contribution.

Cross-Shard Atomic Trading. The symbol-sharded model excels at processing trades for individual symbols in parallel. However, it does not natively support atomic swaps or complex trades that involve multiple symbols simultaneously. Executing such trades would require a cross-shard transaction protocol. We propose investigating the implementation of a two-phase commit (2PC) or optimistic cross-shard transaction mechanism to enable atomic settlement across different symbol shards, a significant challenge in distributed ledger systems [49, 22].

Generalizability to Other DLT Platforms. Our experiments were conducted on a private, IBFT 2.0-based Ethereum network. To validate the generalizability of our architectural principles, future work should include deploying and benchmarking the same architecture on other leading permissioned DLTs, most notably Hyperledger Fabric. Given Fabric's different architectural and consensus model, a comparative analysis would provide crucial insights into how platform choice interacts with application-level design for high-performance workloads [46, 6].

Dynamic Resource and Workload Management. The current system assumes a relatively static set of trading symbols and workloads. A more robust, production-ready system would need to dynamically adapt to changing market conditions, such as sudden spikes in volume for a particular symbol or the introduction of new trading pairs. Future research could focus on developing adaptive resource allocation algorithms that can dynamically scale worker threads or rebalance sharding strategies in real-time.

Enhanced Privacy and Confidentiality. While our system operates on a permissioned network, all settled trades are recorded transparently on the ledger. For many enterprise use cases, confidentiality of trade data is a strict requirement. Future iterations should explore the integration of privacy-enhancing technologies, such as zero-knowledge proofs (e.g., zk-SNARKs) or trusted execution environments (TEEs), to enable confidential transactions without sacrificing auditability [8].

Acknowledgments. This work is supported in part by Horizon Globex Ireland and by Science Foundation Ireland grant 13/RC/2094_P2 to Lero, the Science Foundation Ireland Research Centre for Software.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alchemy: Understanding and handling nonce in EVM chains (2024), https://docs.tatum.io/docs/nonce-what-is-it-and-optional-use, accessed: 2026-01-27
2. Alipanahloo, Z., Hafid, A.S., Zhang, K.: Maximum extractable value (MEV) mitigation approaches in Ethereum and layer-2 chains: A comprehensive survey. IEEE Access **12**, 185212–185231 (2024). https://doi.org/10.1109/ACCESS.2024.3000000
3. Allman, E., Allman, M., Falk, A., Paxson, V., Salt, J.: Pitfalls for testbed evaluations of internet systems. ACM SIGCOMM Computer Communication Review **40**(3), 44–49 (2010)
4. Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: Proceedings of the Spring Joint Computer Conference. pp. 483–485 (1967)
5. Auer, R., Frost, J., Pastor, J.M.V.: Miners as intermediaries: Extractable value and market manipulation in crypto and DeFi. Tech. Rep. BIS Bulletin No. 58, Bank for International Settlements (2024)
6. Baliga, A., Solanki, S., Verekar, S., Kamat, P.: Performance characterization of Hyperledger Fabric. In: 2018 Crypto Valley Conference on Blockchain Technology (CVCBT) (2018)
7. Barroso, L.A., Dean, J., Hölzle, U.: Web search for a planet: The google cluster architecture. IEEE Micro **23**(2), 22–28 (2003)
8. Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from Bitcoin. In: 2014 IEEE Symposium on Security and Privacy. pp. 459–474 (2014)
9. Bez, M., Fornari, G., Vardanega, T.: The scalability challenge of Ethereum: An initial quantitative analysis. In: 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE). pp. 1–10 (2019). https://doi.org/10.1109/SOSE.2019.00010
10. Bilokon, P., Gunduz, B.: C++ design patterns for low-latency applications including high-frequency trading (2023)
11. Cederman, D., Gidenstam, A., Ha, P.: Lock-free concurrent data structures. ArXiv preprint arXiv:1302.2757 (2013)
12. Chainlink Labs: Hybrid smart contracts: Combining on-chain and off-chain computation. Chainlink Research Reports (2024)
13. Credit Suisse Research: Blockchain scalability: A comparative analysis of layer 1 throughput. Tech. rep., Credit Suisse (2025)
14. Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2), 74–80 (2013). https://doi.org/10.1145/2408776.2408794
15. Financial Crimes Enforcement Network (FinCEN): Know your customer (KYC) and anti-money laundering (AML) compliance guidance. U.S. Department of the Treasury (2024)
16. Force, F.A.T.: FATF guidance on virtual assets and virtual asset service providers. Tech. rep., FATF (2024), updated guidance on KYC/AML requirements for blockchain systems

17. Fynn, E., Pedone, F.: Challenges and pitfalls of partitioning blockchains. In: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W). pp. 1–6 (2018). https://doi.org/10.1109/DSN-W.2018.00028
18. Grimmelmann, J.: Regulatory implications of MEV mitigations. Available at SSRN (2024)
19. Gustafson, J.L.: Reevaluating Amdahl's law. Communications of the ACM **31**(5), 532–533 (1988)
20. Harchol-Balter, M.: Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge University Press (2013)
21. Harris, T.L.: A pragmatic implementation of non-blocking linked-lists. In: Proceedings of the 15th International Conference on Distributed Computing. pp. 300–314 (2001)
22. Harris, T.L., Fraser, K.: A lock-free cross-shard transaction protocol. In: Proceedings of the 16th International Symposium on Distributed Computing (2002)
23. Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann, 6th edn. (2017)
24. Hyperledger Foundation: Hyperledger fabric documentation (2024), https://hyperledger-fabric.readthedocs.io/, accessed: 2026-01-26
25. Jamithireddy, N.H., et al.: Hybrid on-chain and off-chain architectures for secure, decentralized fintech payment processing in SAP. In: 2025 International Conference on Next Generation Computing Systems (ICNGCS). IEEE (2025). https://doi.org/10.1109/ICNGCS64900.2025.11182978
26. John, L.K.: Performance evaluation: Techniques, tools and benchmarks. Tech. rep., Electrical and Computer Engineering Department, The University of Texas at Austin (2007)
27. Knuth, D.E.: The Art of Computer Programming, Volume 1: Fundamental Algorithms. Addison-Wesley (1968)
28. Kudzin, A., Toyoda, K., Takayama, S., Ishigame, A.: Scaling Ethereum 2.0's cross-shard transactions with refined data structures. Cryptography **6**(4), 57 (2022). https://doi.org/10.3390/cryptography6040057
29. Kuhn, R., Yaga, D., Voas, J.: Rethinking distributed ledger technology. Tech. Rep. NIST Internal Report 8202, National Institute of Standards and Technology (2019)
30. Lewis, R., McPartland, C., Ranjan, R.: Blockchain and financial market innovation. Economic Perspectives **41**(7) (2017)
31. Little, J.D.C.: A proof for the queuing formula: L = λW. Operations Research **9**(3), 383–387 (1961)
32. Mancino, D., Leporati, A., Viviani, M., Denaro, G.: Decentralization or favoritism? an analysis of Ethereum transactions and maximal extractable value strategies. In: 2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC) (2025)
33. Mazzola, F., Spychiger, F., Tessone, C.J.: Regulated DeFi: The case for compliant decentralized finance. Frontiers in Blockchain **3**, 54 (2020). https://doi.org/10.3389/fbloc.2020.00054
34. Meyes, R., Lu, M., de Puiseau, C.W., Meisen, T.: Ablation studies in artificial neural networks (2019)
35. Moir, M., Shavit, N.: Concurrent data structures. In: Handbook of Data Structures and Applications, pp. 1–48. Chapman and Hall/CRC, 2nd edn. (2018)
36. Network, A.: Layer 2 scaling solutions: Performance benchmarks and comparisons. Technical Documentation (2023)

37. Ng, H., Malik, J.K.: Improving Performance of a Trading System through Lock-Free Programming. Master's thesis, KTH Royal Institute of Technology (2018)
38. O'Hara, M.: Market Microstructure Theory. Blackwell Publishers (1995)
39. Paimani, K.: SonicChain: A wait-free, pseudo-static approach toward concurrency in blockchains (2021)
40. Qin, K., Zhou, L., Gervais, A.: Quantifying and mitigating MEV in layer-2 systems. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (2022)
41. Rahouti, M., Xiong, K., Ghani, N.: Bitcoin concepts, threats, and machine-learning security applications. IEEE Access **6**, 67189–67205 (2018)
42. Shadish, W.R., Cook, T.D., Campbell, D.T.: Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Houghton Mifflin (2002)
43. Song, H., Qu, Z., Wei, Y.: Advancing blockchain scalability: An introduction to layer 1 and layer 2 solutions. In: 2024 IEEE 2nd International Conference on Blockchain and Cryptocurrency (2024)
44. The Wolfsberg Group: Anti-money laundering principles for correspondent banking. Wolfsberg Group Standards (2025)
45. Thompson, M., Farley, D., Barker, M., Gee, P., Stewart, A.: LMAX disruptor: High performance inter-thread messaging library (2011), https://lmax-exchange.github.io/disruptor/, accessed: 2025-12-01
46. Ucbas, Y., Eleyan, A., Hammoudeh, M., Alohaly, M.: Performance and scalability analysis of Ethereum and Hyperledger Fabric. IEEE Access **11**, 1–15 (2023). https://doi.org/10.1109/ACCESS.2023.3000000
47. Wang, Y., et al.: Understanding Ethereum mempool security under asymmetric DoS by symbolized stateful fuzzing. In: Proceedings of the 33rd USENIX Security Symposium (2024)
48. Xu, J., Paruch, K., Cousaert, S., Feng, Y.: SoK: Decentralized exchanges (DEX) with automated market maker (AMM) protocols. ACM Computing Surveys **56**(5), 1–37 (2023). https://doi.org/10.1145/3570639
49. Zhang, J., et al.: Front-running attack in distributed sharded ledgers and a countermeasure (2023)